

Project 3: Designing and Optimizing a Reliable Data Transmission Protocol

1 Project Goals

In this project, you will design, implement, and optimize reliable data transfer protocols. A basic requirement of any reliable transfer protocol is **correctness**. Correctness entails that although the network itself may drop and reorder packets, application data such as a file should be transferred in order and without loss. However, an equally important consideration is **performance**.

Consider a file of size F bytes transferred over a network with a bandwidth B Mbps. Owing to the need to wait for acknowledgments (ACKs), and if necessary, retransmit data, it is unlikely that any protocol can reliably transfer data at a rate of B Mbps. We refer to the ratio of the useful data transferred (F bytes) to the total transfer time as the **goodput** achieved by the protocol. Further, a reliable transfer protocol incurs **overheads**, which we define as $T - F$, where T is the total bytes transmitted by the sender or receiver over the network inclusive of ACKs, retransmitted packets, and header fields.

In this project, your goal is to **design and implement a reliable transfer protocol that can achieve high goodput, and low overhead under diverse network conditions**. You will compare your protocol with a baseline scheme for comparison, which you will also implement. While we provide hints, the specific optimizations that you will implement in your protocol is up to you. While correctness of all implementations is a minimal, mandatory requirement, the project will be primarily evaluated on

- the kinds of optimizations you implement.
- the performance that you achieve.
- a documentation of the performance of the schemes under different network conditions, along with a clear understanding of the design trade-offs.

You will document the above information in a report, which is also a mandatory requirement for the project.

2 Requirements

You will implement the following protocols:

- **(Baseline Scheme) Stop and Go:** In this protocol, the sender sends a packet and waits for an ACK before sending the next packet. If the corresponding ACK is not received within a predefined timeout period, the packet is retransmitted. This protocol results in a low overhead but significantly lower goodput.
- **Your own optimized protocol:** You are free to design or optimize your protocol as you choose, but with a view point of achieving high goodput under diverse conditions, and low overheads

We next discuss a strategy that could serve as a starting point. Here, the sender maintains a sliding window of packets to be transmitted to the receiver, moving the window forward as it receives ACKs. The sender transmits W packets to the receiver, where W is the window size. The receiver uses a

cumulative ACK scheme, and retransmits on a timeout. For example, consider that when transmitting a window of 5 packets, packet 3 is dropped but packets 1, 2, 4, 5 are successfully delivered. The receiver sends an ACK for packet 2 multiple times (when each of packets 2, 4, and 5 are received). Under this strategy, the sender will eventually timeout on packet 3, and retransmit packets 3, 4, and 5.

However, the above should only serve as a starting point, and we expect the best projects would include many other optimizations. Some hints to consider:

- Does a sender need to wait until a timeout to detect packet loss, or are there cases it can retransmit earlier?
- Are there disadvantages of a cumulative ACK scheme, and how could one do better? (hint: selective ACKs).
- What is a good window size to use and how would this depend on network settings such as bandwidth and delay?
- What is a good choice of timeout? (hint: ideally, the timeout should match the round-trip time of the network, but it is desirable to leave some slack for delay variability. The amount of slack may need to be experimentally tuned and reported).

3 Reliable Transport Simulator

We provide code for

1. a network emulator, which emulates network conditions based on settings specified in a configuration file.
2. a module `monitor.py` which interfaces with the network emulator that must be called by your sender and receiver code.

The starter code is located in the `Student Code/example` directory.

3.1 Configuration File and Network Parameters

The network parameters are specified in a configuration file (`config.ini`), which can be tweaked to emulate a real-world scenario. The network emulator reads the configuration file (`config.ini`) and sets the various parameters.

The configuration file contains the host and port information for all endpoints (we will use the term endpoint to refer generically to the sender and receiver) involved. All parameters of the configuration file should be set before launching the NE or the endpoints. The NE reads the host/port information to track senders and forwards messages to the correct receivers.

The Network Monitor (see 3.3) reads the configuration file, binds to the appropriate port, and connects to the NE using its relevant host/port information. The configuration file also contains necessary filenames (e.g., file being transmitted, file used for logging etc.). Each endpoint (Sender/Receiver) should instantiate a Monitor object and pass the config file path and the correct header.

Your code should run based on the parameters in the `NODES` section of the config file. However, **your code could (and should!) also read the network parameters in this file so that it can set the window size and other parameters appropriately.**

The configuration file further contains the fields for the file being sent (`file_to_send`) and the output file (`write_location`) on the receiver end. These fields should be read by the sender and

receiver to be able to transmit and save the file. See References and Useful Information (6) to understand how to read the config file.

3.1.1 Network Parameters

- `PROP_DELAY` (Propagation Delay) is the time it takes for the packet to travel from the sender to the receiver. As you learned in the network performance unit, this is separate from the transmission delay. This field is in seconds.
- `MAX_PACKET_SIZE` is the maximum payload/data transferred per packet. In Figure 2, the combined header + data should be less than or equal to the `MAX_PACKET_SIZE`. Larger packet sizes are not accepted and dropped.
- `LINK_BANDWIDTH` is the link bandwidth at which the packets are pushed onto the link (in this case simulated by the NE). Transmission delay is the size of the packet divided by the link bandwidth. This field is in bytes per second (BPS).
- `MAX_PACKETS_QUEUED` gives the upper limit on the number of outstanding packets that can be queued in the NE. Any new packets are dropped. This value is set using the bandwidth-delay product of the network.
- `DROP_MODE` sets the pattern used to generate losses by the NE. `DROP_MODE=1` results in packets being dropped randomly (the probability of drop is set based on `RANDOM_DROP_PROBABILITY`). `DROP_MODE=2` is a setting where packets are dropped with a probability based on the queue length in a router buffer that the NE emulates. For this project, only `DROP_MODE=1` will be used.
- `RANDOM_DROP_PROBABILITY` sets the probability at which packets are dropped by the NE in `DROP_MODE=1`. If this field is set to 0 then no drops occur. For a drop probability in $(0, 1]$, the drops occur at the probability value specified.
- `REORDER_PROBABILITY` sets the probability at which packets are reordered by the NE. If this field is set to 0 then no reorders occur. For $(0, 1]$, the reorders occur at the probability value specified. The reordered packets are inserted back into the internal NE queue to be transmitted later.

Note: A packet is transmitted by the NE between the endpoints solely based on the destination ID given. The network emulator doesn't check or modify the packet content in any way. The packet can contain any number of header fields and in any format (there is no need to comply with TCP's header format). The only constraint is the `MAX_PACKET_SIZE` parameter mentioned above.

3.2 Network Emulator (NE)

The NE emulates an unreliable data transfer layer which can cause **data loss, and reordered packets**. A visualization of this can be seen in Figure 1. The NE is also responsible for enforcing various parameters such as the propagation delay and bandwidth of the network (see Network Parameters for a detailed list). The parameters are set using the configuration file (.ini format).

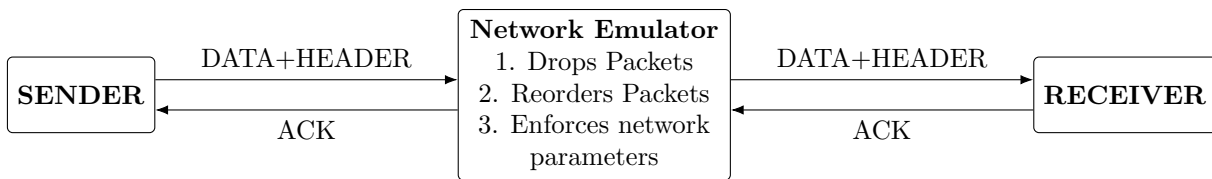


Figure 1: Emulator setup and functions.

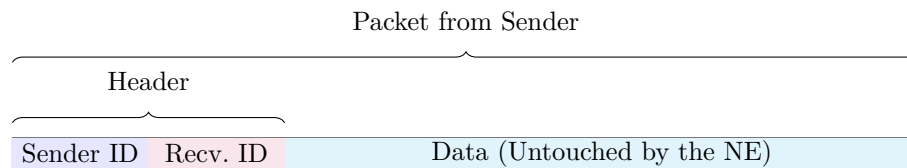


Figure 2: Packet format required by the network emulator.

3.3 Network Monitor

The network monitor runs alongside the endpoints and provides the interface to send/receive packets, as well as to indicate when the transmission is complete. The monitor also extracts network information such as data overhead, goodput, and the number of packets transmitted. The network monitor on the receiver side checks for the correctness of the transmission as well.

The Python module `monitor.py` should be included in each of your endpoint programs. Copy `monitor.py` to the same directory as your endpoint code. Information on how to interface with the Monitor is provided below:

- Initialize a Monitor object with the following parameters:
 - `configFile` - path to the Configuration file
 - `config_heading` - Defines the section to read the fields from. Either 'sender' or 'receiver' based on whether the endpoint is a sender or a receiver.
- Below are the ways that your sender and receiver can use the Monitor object that you created:
 - Send/receive packets using the monitor (which handles communication with the NE): Call `Monitor.send(dest, data)` to send a packet. This sends the packet without modifying it to the network emulator to be forwarded to the destination. This calls `socket.sendto()` in the background.
 - Call `Monitor.recv(Bytes)` to receive an incoming packet. This calls `socket.recvfrom()` in the background.
 - Call `Monitor.send_end(dest_id)` after the transmissions to the endpoint are complete. **Monitor.send_end() is to be called after the sender knows that the receiver has received the final packet (for example, after you receive the ACK for the final packet in Stop and Go protocol). This is only relevant to the sender.**
 - Call `Monitor.recv_end(recv_file, sender_id)` once the entire file is received and written to `recv_file`. This will check for the correctness of the file. This is only relevant for the receiver.

3.4 Viewing Performance Statistics

After the file has been successfully transferred and `send_end()` and `recv_end()` functions are called, the monitor prints out the performance statistics to `sender_monitor.log` and `receiver_monitor.log`.

3.4.1 Sender_monitor.log

An example sender log is shown below:

```
1 1617394269.3555202
2 Id:1 | Starting monitor on ('localhost', 8001).
3
4 File Size           : 628370 bytes
5 Total Bytes Transmitted : 659730 bytes
6 Overhead            : 31360 bytes
7 Number of Packets sent   : 663
8 Total Time           : 162.87 secs
9 Goodput              : 3858.15 bytes/sec
```

Here, Line 1 is the timestamp. Overhead is simply the extra bytes transmitted beyond the actual file size, including the header per packet and retransmitted packets, while Goodput is the effective throughput which is the useful bytes (file size) transmitted divided by the total time taken. You should primarily be reporting the Overhead and Goodput numbers in your report.

3.4.2 Receiver_monitor.log

An example receiver log is shown below:

```
1 1617394268.3460567
2 Id:2 | Starting monitor on ('localhost', 8002).
3
4 File transmission correct : True
5 Number of Packets Received : 649
6 Total Bytes Transmitted   : 645761
7 Total Time                : 162.66 secs
```

Here, Line 1 is the timestamp, while Line 4 indicates whether the transmitted file was received reliably or not. Note that there is a slight difference between the bytes and packets reported at the sender and receiver owing to packet drops. The total time is also slightly different owing to differences in when timers are started and stopped. Please report the numbers in the sender log in your report.

4 What You Need To Do

For this project, you are required to implement two different reliable transport protocols by writing the sender and receiver code for each of them:

1. Write sender and receiver programs using the Stop and Go Protocol. Place your code in a directory named `Student Code/stop_and_go`. Provide a Makefile in this directory that supports `make run-sender config=<configfile>` and `make run-receiver config=<configfile>` to run the sender and receiver codes with the supplied configuration file.
2. Write sender and receiver code based on the protocol that you have designed. Place your code in a directory named “Student Code/student”. Provide a Makefile in this directory compatible with ‘`make run-sender config=<configfile>`’ and ‘`make run-receiver config=<configfile>`’ to run the sender and receiver codes with the supplied configuration file. (See BASH command-line

arguments).

The test config files stress different aspects of the implemented protocols.

Note 1: The performance of the above protocols depends on the choice of the timeout value and the choice of window size, which itself needs to be chosen in a configuration-specific way. Please make sure to set the timeout and window size automatically to values that are reasonable for different configurations using good rules of thumb.

Note 2: Since we are emulating the data transfer delays using the emulator, note that for higher bandwidth and larger window sizes, the performance of the emulator significantly degrades. To avoid this, we must limit the window size to 50 in all cases. Additionally, if you're using a NACK scheme, your buffer can't be larger than the maximum window size (50), and you aren't allowed to send the entire file at once.

Note 3: If your Python code is excessively slow in critical paths, you may encounter significant delays in packet transmission/receipt that affects your final goodput. To avoid this, we recommend avoiding unnecessary operations in time-sensitive portions of your code. The biggest offender here are print/log statements: Avoid printing/logging in any time-sensitive sections of your code.

4.1 Steps your code should follow

In all cases, your code should follow these steps in order to ensure proper functionality with the Network Emulator and Network Monitor:

1. Initialize the sender and receiver network monitors.
2. Start the receiver. After a short delay (around 1 second), start the sender.
3. Exchange messages through `Monitor.send()` and `Monitor.recv()`.
4. When the receiver receives the final packet, call `Monitor.recv_end()`. **IMPORTANT: Leave the receiver running and sending ACKs to any additional packets that may arrive!**
5. When the sender receives an ACK for the final packet, call `Monitor.send_end()`. This will shut down the network emulator.

The order of steps 4 and 5 are important to ensure proper behavior. Consider the case where the sender sends the last packet and the receiver's ACK is dropped: If the receiver has quit, the sender will continue retransmitting the packet forever and getting no response. Therefore, the receiver should continue running and ACKing any retransmitted packets. Likewise, the sender should not call `send_end()` or quit until it receives the ACK for the final packet. If it quits prematurely, it can not be certain that the receiver has gotten all packets.

4.2 Running your code

We will be testing your code using the configuration files similar to the ones provided under `TestConfig` in the starter code. The network emulator will be launched before the endpoints. Then the endpoints will be launched. Here, `config.ini` is the command-line parameter passed to the Make run commands.

5 Submission and Grading

5.1 Submitting Your Code

You will need to submit your implementations, as well as a report.

- For code submission, you will need to submit 4 files. Note that the name of each file must match exactly as the one listed below to prevent auto grader failure. Please do not submit zip files or directories.
 1. `receiver.py` (The receiver of your optimized protocol)
 2. `receiver_stop_and_go.py` (The receiver of your stop and go implementation)
 3. `sender.py` (The sender of your optimized protocol)
 4. `sender_stop_and_go.py` (The sender of your stop and go implementation)
- The report based on the provided template, following all the guidelines.

5.2 Project Checkpoints and Deliverables

The project is structured around two major submission checkpoints. Each checkpoint includes both code and written deliverables. Meeting the minimum deliverables is necessary for receiving full credit for that checkpoint. Stronger submissions will exceed these minimum requirements through deeper analysis, clearer presentation, and more thoughtful design choices.

Checkpoint	Minimum Code Deliverables	Minimum Written Deliverables
Interim	- Working implementations of stop and go. - Must pass all correctness checks.	Report with: - Design details of stop and go. - Performance results. - Analysis and observations based on these results. - An initial proposal for a new protocol (ideas and motivation are sufficient; implementation is not required at this stage. You will not be penalized for changing or refining your approach in the final submission).
Final	- Final implementation of your custom protocol. - Must pass all correctness checks. - Goodput and overhead must be optimized as much as possible, and meet the minimum requirements	Polished final report with: - Final description of your protocol, including all optimizations made. - Final performance results updated along with necessary graphs. - Comparisons between your protocol and stop and go. - Analysis and observations.

5.3 Grading

A minimal expectation for all protocol implementations is correctness. A major criterion for grading is your performance results, whether the overall trends make sense, and the effort and creativity you have shown in adding optimizations. The clarity of report (documenting the effort, appropriately presenting graphs and containing the right level of detail in the writing) will also be a factor.

5.3.1 Grading Breakdown

Implementation and Performance (55%) This portion of the grade evaluates whether the required protocols are correctly implemented, functional in the provided simulator, and meet established performance requirements. The implementation score is broken down as follows:

- Interim submission: Stop and Go correctness check (15%):** The interim submission is intended to verify that your implementation of stop and go is correct and functional. At this stage, correctness and successful execution are the only primary goals. Your code should run on the provided simulator without errors and produce sensible output.
- Final submission: custom protocol performance (40%):** For the final submission, your custom protocol must compile and run correctly in the autograder environment. We only test with 2% loss and re-ordering, and use the large file. For the **grading baseline**, we use a goodput

of 100,000 bytes/s, with an overhead of 30,000 bytes. An initial raw score is calculated using the following formula:

$$\text{Raw Score: } 100 + 50 * (\text{Goodput Ratio} - \text{Overhead Ratio})$$

$$\text{Goodput Ratio: } \frac{\text{Student Goodput}}{\text{Baseline Goodput}}$$

$$\text{Overhead Ratio: } \frac{\text{Student Overhead}}{\text{Baseline Overhead}}$$

The raw score is constrained to the range of $[0, 100]$. That is, if your raw score is greater than 100, your raw score is capped at 100. If your raw score is less than 0, your raw score will be adjusted to 0.

We then calculate the code performance score by using the formula:

$$\text{Performance score: } 65 + 35 * \frac{\text{Adjusted Raw Score}}{100}$$

Note: The performance benchmark we use are a bit weaker than the TA solutions we reported on the performance sheet. Further, the formula captures the idea that for a solution with 10% higher goodput, we will accept 10% higher overheads.

Interim Submission Report (5%) Your interim report will be graded on completeness, and if it includes the minimum written deliverables as outlined in Section 5.2

Final Report (40%) The final report is evaluated along **Design**, **Evaluation**, and **Quality**. Each component is scored independently using the rubric below.

Design (20%)

Performance Level	Points	Criteria
Excellent	20	• More than two interesting features. • Schemes must be explained clearly.
Good	18	• More than two schemes with poor explanation.
Acceptable	16	• Only two interesting features with clear explanations.
Passable	14	• Only two schemes with poor explanation.
Below Expectations	10	• One or less interesting feature, or used infinite client buffer.

What qualifies as interesting features?

- For the acceptable grade, typically cumulative and selective ACK are the two interesting features.
- For additional features over and beyond, some examples are dynamic timeout, fast retransmit, dynamic window size.
- Some examples that **don't qualify** as interesting: adjusting timeout or window size statically based on config size alone, or implementation artifacts (e.g, related to # of threads, etc.).

Ablation Study (Evaluation) (10%) Did you systematically evaluate your features? Did you explain your findings well?

Performance Level	Points	Criteria
Excellent	10	• More than two interesting features. • Proper ablation study
Acceptable	7	• Only two interesting features, but with a proper ablation study.
Passable	5	• Only one interesting feature, with a proper ablation study.
Below Expectations	3	• One or less interesting feature, or used infinite client buffer. • Did not perform a proper ablation study.

Figures and Report Quality (10%) Did you include mean and variance graphs for stop and go versus your protocol?

Report quality is important and is viewed as an indicator of effort. For instance, it should be very clear what configuration you are reporting on. Graphs should be clear and legible. These are subjective factors that may impact your grade.

Performance Level	Points	Criteria
Excellent	10	- Variance Plot - Plot included - Axes labelled - Report Quality - Protocol explanation present - Feature summary present - Required Experiments Present - Stop and Go - Window size - Your protocol vs stop and go - Sensitivity testing - Discussion Present - Interim discussion - Changes summary - Final discussion
Acceptable	7	1 red category miss + ≤ 1 blue category miss 0 red category miss + 2 blue category miss
Passable	5	2 red category miss + ≤ 1 blue category miss 1 red category miss + 2 blue category miss 0 red category miss + $\geq 3, \leq 5$ blue category miss
Below Expectations	3	Everything else

5.3.2 Extra Credit

For extra credit, we will evaluate your implementations under exceptionally strange network conditions. We will not reveal these situations, as we are testing the robustness under unexpected conditions. As a result, if you would like, it is up to you to modify the network emulator to implement strange network conditions and test robustness. Note that for full credit without the extra credit, you will not need to do such modifications, merely passing the requirements is enough. The extra credit will only be evaluated for the final checkpoint.

6 References and Useful Information

The configuration file is to be extracted and read using the `configparser` library. See the link below for more information. Using pip, `configparser` can be installed with the command

```
pip3 install --user configparser
```

<https://docs.python.org/3/library/configparser.html>